

Competitive Programming

Section 6 Editorial: Two pointers and sliding windows

Competitive Programming Course

2026-07-15

Table of contents

1	How to read this editorial	4
2	Problem A: Sorted Two Sum	5
2.1	Restatement	5
2.2	Important constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	5
2.5	Algorithm	5
2.6	Correctness	6
2.7	Complexity	6
2.8	C++ solution	6
2.9	Python solution	7
2.10	Common wrong answers	8
3	Problem B: Longest Bounded Window	9
3.1	Restatement	9
3.2	Important constraints and traps	9
3.3	Observations	9
3.4	Sample walk-through	9
3.5	Algorithm	9
3.6	Correctness	10
3.7	Complexity	10
3.8	C++ solution	10
3.9	Python solution	11
3.10	Common wrong answers	11
4	Problem C: Count Subarrays At Most K	13
4.1	Restatement	13
4.2	Important constraints and traps	13
4.3	Observations	13
4.4	Sample walk-through	13
4.5	Algorithm	14
4.6	Correctness	14
4.7	Complexity	14
4.8	C++ solution	14
4.9	Python solution	15
4.10	Common wrong answers	16
5	Problem D: Cover All Labels	17
5.1	Restatement	17
5.2	Constraints and traps	17
5.3	Observation and trace	17
5.4	Algorithm	17
5.5	Correctness proof	18

5.6	Complexity	18
5.7	Full C++ solution	19
5.8	Full Python solution	19
5.9	Implementation traps and tests	20
6	Stress testing strategy	21
7	What to carry forward	22

1 How to read this editorial

Two-pointer solutions are short, but the proof is the point. Every time a pointer moves, ask:

What candidates did we just discard, and why can none of them be part of a better answer?

For sliding windows, also ask why the left pointer never needs to move backward. In this section, the reason is that all array values are nonnegative.

2 Problem A: Sorted Two Sum

2.1 Restatement

Given a sorted array, determine whether two distinct elements sum to k .

2.2 Important constraints and traps

The array is already sorted. You must use two distinct indices, so the loop condition is `left < right`.

Values may be negative, but sorted two-sum still works because the array order is monotone.

2.3 Observations

Let $sum = a[left] + a[right]$.

- If $sum == k$, a pair exists.
- If $sum < k$, then $a[left]$ is too small even with the largest available partner. Move `left` right.
- If $sum > k$, then $a[right]$ is too large even with the smallest available partner. Move `right` left.

2.4 Sample walk-through

Array:

1 2 4 7 11

Target 9.

left	right	sum	action
1	11	12	move right
1	7	8	move left
2	7	9	found

2.5 Algorithm

```

left = 0
right = n - 1
while left < right:
    total = a[left] + a[right]
    if total == k: YES
    if total < k: left += 1
    else: right -= 1
NO

```

2.6 Correctness

At each step, all possible pairs using indices outside `[left, right]` have already been safely discarded.

If $a[\text{left}] + a[\text{right}] < k$, then for every $j \leq \text{right}$:

$$a[\text{left}] + a[j] \leq a[\text{left}] + a[\text{right}] < k.$$

So $a[\text{left}]$ cannot form a valid pair with any remaining element. Moving `left` discards no solution.

If $a[\text{left}] + a[\text{right}] > k$, then for every $i \geq \text{left}$:

$$a[i] + a[\text{right}] \geq a[\text{left}] + a[\text{right}] > k.$$

So $a[\text{right}]$ cannot form a valid pair with any remaining element. Moving `right` discards no solution.

If a valid pair exists, it remains inside the active range until it is found. If the pointers cross, no pair remains. Therefore the algorithm is correct.

2.7 Complexity

Each pointer moves at most n times. Time is $O(n)$ and memory is $O(1)$.

2.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
}

```

```

while (T--) {
    int n;
    long long target;
    cin >> n >> target;
    vector<long long> values(n);
    for (long long &value : values) cin >> value;

    int left = 0;
    int right = n - 1;
    bool ok = false;
    while (left < right) {
        long long total = values[left] + values[right];
        if (total == target) {
            ok = true;
            break;
        }
        if (total < target) ++left;
        else --right;
    }
    cout << (ok ? "YES" : "NO") << '\n';
}
return 0;
}

```

2.9 Python solution

```

import sys

def exists_pair(values: list[int], target: int) -> bool:
    left = 0
    right = len(values) - 1
    while left < right:
        total = values[left] + values[right]
        if total == target:
            return True
        if total < target:
            left += 1
        else:
            right -= 1
    return False

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):

```

```
n = data[idx]
target = data[idx + 1]
idx += 2
values = data[idx:idx + n]
idx += n
out.append("YES" if exists_pair(values, target) else "NO")
sys.stdout.write("\n".join(out))
```

2.10 Common wrong answers

- Using `left <= right`, which can use the same element twice.
- Moving the wrong pointer after comparing the sum.
- Forgetting duplicate values can form a valid pair if they are at distinct indices.

3 Problem B: Longest Bounded Window

3.1 Restatement

Find the maximum length of a contiguous subarray with sum at most k . Values are nonnegative.

3.2 Important constraints and traps

The empty subarray is allowed, so if every single value is greater than k , the answer is 0.

The nonnegative constraint is essential. Without it, this sliding-window solution is not valid.

3.3 Observations

Maintain a window `[left, right]` and its sum. Add each new `right` value. If the sum becomes too large, move `left` right until the sum is valid again.

Because values are nonnegative, removing values from the left never increases the sum.

3.4 Sample walk-through

Array:

2 1 3 4 2

Limit 7.

right	added	after shrinking	sum	best
0	2	[0,0]	2	1
1	1	[0,1]	3	2
2	3	[0,2]	6	3
3	4	[2,3]	7	3
4	2	[3,4]	6	3

3.5 Algorithm

```

left = 0
sum = 0
best = 0
for right from 0 to n - 1:
    sum += a[right]
    while sum > k:
        sum -= a[left]
        left += 1
    best = max(best, right - left + 1)
print best

```

3.6 Correctness

After the shrinking loop finishes for a fixed `right`, the window sum is at most `k`, so `[left, right]` is valid. Since values are nonnegative, any start position smaller than `left` would include all elements of the current window plus some removed nonnegative values; it cannot have smaller sum. Thus no valid window ending at `right` starts before `left`.

Therefore `[left, right]` is the longest valid window ending at `right`. Taking the maximum of these lengths over all `right` gives the longest valid window overall.

3.7 Complexity

Both pointers move only forward. Time is $O(n)$ and memory is $O(1)$.

3.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        long long limit;
        cin >> n >> limit;
        vector<long long> values(n);
        for (long long &value : values) cin >> value;

        int left = 0;
        long long total = 0;
        int best = 0;
    }
}

```

```

    for (int right = 0; right < n; ++right) {
        total += values[right];
        while (left <= right && total > limit) {
            total -= values[left];
            ++left;
        }
        best = max(best, right - left + 1);
    }
    cout << best << '\n';
}
return 0;
}

```

3.9 Python solution

```

import sys

def longest_window(values: list[int], limit: int) -> int:
    left = 0
    total = 0
    best = 0
    for right, value in enumerate(values):
        total += value
        while left <= right and total > limit:
            total -= values[left]
            left += 1
        best = max(best, right - left + 1)
    return best

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    limit = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(longest_window(values, limit)))
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Using `if total > k` instead of `while total > k`.

- Forgetting the answer can be zero.
- Applying this method to arrays with negative values.
- Updating **best** before shrinking to a valid window.

4 Problem C: Count Subarrays At Most K

4.1 Restatement

Count nonempty contiguous subarrays with sum at most k . Values are nonnegative.

4.2 Important constraints and traps

The answer may be $n(n+1)/2$, so use 64-bit integers in C++.

The task asks for count, not maximum length. For each right endpoint, many subarrays may be valid.

4.3 Observations

Use the same valid window $[\text{left}, \text{right}]$ as in Problem B. After shrinking, all starts from left through right form valid subarrays ending at right .

There are:

```
right - left + 1
```

of them.

4.4 Sample walk-through

Array:

```
1 2 3
```

Limit 3.

right	valid starts	new subarrays	count
0	0	[1]	1
1	0, 1	[1,2], [2]	3
2	2	[3]	4

Answer: 4.

4.5 Algorithm

```
left = 0
sum = 0
answer = 0
for right from 0 to n - 1:
    sum += a[right]
    while sum > k:
        sum -= a[left]
        left += 1
    answer += right - left + 1
print answer
```

4.6 Correctness

After shrinking, `[left, right]` has sum at most `k`, and no window ending at `right` with start before `left` is valid. Since values are nonnegative, any subarray ending at `right` and starting at a position `s` with `left <= s <= right` has sum no greater than the sum of `[left, right]`, so it is valid.

There are exactly `right - left + 1` such starts. The algorithm adds that number, so it counts exactly all valid subarrays ending at `right`.

Every nonempty subarray has exactly one right endpoint. Summing the counts over all right endpoints counts every valid subarray exactly once.

4.7 Complexity

Each pointer moves at most `n` times. Time is $O(n)$, memory is $O(1)$.

4.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        long long limit;
        cin >> n >> limit;
        vector<long long> values(n);
```

```

    for (long long &value : values) cin >> value;

    int left = 0;
    long long total = 0;
    long long answer = 0;
    for (int right = 0; right < n; ++right) {
        total += values[right];
        while (left <= right && total > limit) {
            total -= values[left];
            ++left;
        }
        answer += right - left + 1;
    }
    cout << answer << '\n';
}
return 0;
}

```

4.9 Python solution

```

import sys

def count_subarrays(values: list[int], limit: int) -> int:
    left = 0
    total = 0
    answer = 0
    for right, value in enumerate(values):
        total += value
        while left <= right and total > limit:
            total -= values[left]
            left += 1
        answer += right - left + 1
    return answer

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    limit = data[idx + 1]
    idx += 2
    values = data[idx:idx + n]
    idx += n
    out.append(str(count_subarrays(values, limit)))
sys.stdout.write("\n".join(out))

```

4.10 Common wrong answers

- Adding only 1 for each valid right endpoint.
- Forgetting zero-valued elements create many valid subarrays.
- Using 32-bit `int` for the answer.
- Using this method with negative values.

5 Problem D: Cover All Labels

5.1 Restatement

Find the shortest nonempty contiguous segment that contains every label from 1 through m . If at least one label never occurs, the answer is -1 .

This is the section's blind-practice task. It still uses a moving window, but the window condition is coverage rather than a numeric sum.

5.2 Constraints and traps

The total array length is 200000, so checking every pair of endpoints is too slow. Labels are bounded by m , making a frequency array of size $m+1$ appropriate. A window containing many copies of one label is not complete unless it contains every *different* required label.

5.3 Observation and trace

Keep a window and a counter **missing**: the number of labels that occur zero times in the window. Adding a label whose current count is zero decreases **missing**. Removing a label whose count becomes zero increases it.

For 1 2 1 3 2 with labels 1,2,3:

right value	window after expansion/shrinking	missing	best
1	[1]	2	none
2	[1,2]	1	none
1	[1,2,1]	1	none
3	[2,1,3] after removing the first 1	0	3
2	[1,3,2] after removing the first 2	0	3

Once **missing** is zero, the current window is valid. Removing from the left is the only way to make that window shorter; keep doing so until one required label disappears.

5.4 Algorithm

```

count[1..m] = 0
missing = m
left = 0
answer = infinity

for right from 0 to n-1:
    add a[right] to count
    if it was absent before: missing -= 1

    while missing == 0:
        answer = min(answer, right-left+1)
        remove a[left] from count
        if it becomes absent: missing += 1
        left += 1

print answer, or -1 when it was never updated

```

5.5 Correctness proof

Lemma 1. At every point, `missing` equals the number of required labels absent from the current window `[left, right]`.

Proof. Initially every required label is absent. The algorithm changes `missing` exactly when adding the first occurrence of a label or removing its last occurrence. These are exactly the events that change whether a label is absent. $\square$

Lemma 2. Every candidate used to update `answer` contains every required label.

Proof. A candidate is recorded only while `missing == 0`. By Lemma 1, no required label is absent from that window. $\square$

Lemma 3. For each right endpoint, the algorithm records the shortest valid window ending at that endpoint.

Proof. While the window is valid, the algorithm removes leftmost labels one at a time and records each valid length. It stops exactly after removing the last occurrence of one required label, at which point no shorter window with the same right endpoint can be valid. Thus the final recorded valid window for that endpoint is shortest. $\square$

Theorem. The algorithm prints the minimum length of a subarray containing all required labels, or `-1` when none exists.

Proof. By Lemma 2, every recorded length is feasible. Every feasible subarray has some right endpoint; by Lemma 3 the algorithm considers a valid window no longer than that subarray for the same endpoint. Taking the minimum therefore gives the global optimum. If no window is recorded, Lemma 1 shows no window ever contained all labels, so `-1` is correct. $\square$

5.6 Complexity

Each endpoint moves from left to right at most once. Time is $O(n+m)$ per test case and memory is $O(m)$.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    if (!(cin >> t)) return 0;
    while (t--) {
        int n, m;
        cin >> n >> m;
        vector<int> values(n);
        for (int &value : values) cin >> value;

        vector<int> count(m + 1, 0);
        int missing = m;
        int left = 0;
        int answer = n + 1;
        for (int right = 0; right < n; ++right) {
            if (count[values[right]]++ == 0) --missing;
            while (missing == 0) {
                answer = min(answer, right - left + 1);
                if (--count[values[left]] == 0) ++missing;
                ++left;
            }
        }
        cout << (answer == n + 1 ? -1 : answer) << '\n';
    }
    return 0;
}
```

5.8 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 0
    tests = data[index]
    index += 1
    out: list[str] = []
    for _ in range(tests):
```

```

n = data[index]
labels = data[index + 1]
index += 2
values = data[index:index + n]
index += n

count = [0] * (labels + 1)
missing = labels
left = 0
answer = n + 1
for right, value in enumerate(values):
    if count[value] == 0:
        missing -= 1
    count[value] += 1
    while missing == 0:
        answer = min(answer, right - left + 1)
        count[values[left]] -= 1
        if count[values[left]] == 0:
            missing += 1
        left += 1
    out.append(str(-1 if answer == n + 1 else answer))
sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

5.9 Implementation traps and tests

- Count distinct covered labels, not the number of window elements.
- Update the answer before removing the leftmost label.
- Stop shrinking only after a count becomes zero.
- Return -1 when one required label is absent from the whole array.

Test one required label, a missing label, a shortest window at the beginning or end, and windows with many duplicate labels. The random generator uses a quadratic endpoint oracle on small arrays.

6 Stress testing strategy

All three problems have simple slow oracles:

- sorted two-sum: all pairs;
- longest bounded window: all subarrays, keep max valid length;
- count at most k : all subarrays, count valid ones.

The local random tests compare against these slow versions on small arrays. Good tests include duplicates, zeros, limits of zero, all values too large, and arrays where the shrinking loop runs many times.

7 What to carry forward

Two-pointer algorithms are about monotonicity. A pointer move is justified only when it discards candidates that cannot become useful later.

This idea will reappear in:

- binary search predicates;
- greedy algorithms;
- sweep-line active sets;
- offline query processing;
- advanced sliding-window counting problems.